TVL for Tuned Systems

From Agent Building to Agent Engineering

Traigent Labs

January 30, 2026

Preface: A New Chapter in Software History

The Building Blocks Have Changed

For sixty years, software engineering rested on a stable foundation: functions were *well-defined*—whether deterministic or non-deterministic, their behaviour was specified and bounded. They were also *well-scoped*: `sort()`, `predict_house_price()`, `compute_tax()`. Each function did one thing, and you knew exactly what that thing was. Variables held values you assigned, and "correct" meant the same thing today as it did yesterday. That foundation has shifted.

When a function invokes a large language model (LLM)[1], both guarantees collapse. The behaviour becomes *arbitrary*—not merely non-deterministic, but fundamentally unbounded and opaque. The same input may yield different outputs depending on which model the request is routed to, what evidence is retrieved (and how it is chunked, ranked, and packed into context), which tools are available (and their schemas), and which safety, latency, and cost guardrails are active. These are not cosmetic settings—they can be the difference between a grounded answer and a hallucination, a valid completion and an unnecessary refusal, a policy-compliant response and a data-handling incident, a sub-second interaction and a blown SLO, or a $0.01 query and a $0.10 query. A prompt that worked brilliantly last quarter may fail after the next model update. Unlike a random number generator whose distribution is fixed by design, an LLM's output distribution is learned, drifts with every model update, and spans a space too vast to enumerate or test exhaustively.

And the scope? "Review my code." "Plan me a trip to Monte Carlo." "Draft a contract." These are not functions in the classical sense—they are *capabilities* that resist precise specification. The boundary between what the system can and cannot do is no longer a type signature; it is an empirical question answered by evaluation. The variables governing these systems—model choice, temperature, retrieval depth, safety thresholds—are not constants to be hard-coded; they are *tuned variables* that must be revisited whenever the world changes or our preferences evolve.

In an agentic stack, configuration is not "plumbing." It is the control surface of behaviour.

This is not a bug to be patched. It is a fundamental shift in what software *is*.

[1] Or, more generally, an *intelligent callable*: any function, method, or API endpoint backed by a learned model—where behaviour is discovered through evaluation rather than specified by code.

From Building to Engineering

In 1968, NATO convened a conference to address the "software crisis"—projects failing, budgets exploding, systems unreliable. The term *software engineering* was coined to signal that building software required the same rigour as building bridges. Fifty years later, we face a new crisis: AI systems deployed without specification, validated by vibes, promoted by hope.

The numbers are stark. An MIT study finds that 95% of generative AI pilots fail to deliver measurable business impact[2]. S&P Global reports that 42% of companies abandon most AI initiatives before production—up from 17% just one year earlier—with organizations scrapping 46% of proof-of-concept projects on average[3]. Gartner predicts that over 40% of agentic AI projects will be canceled by 2027 due to "escalating costs, unclear business value, or inadequate risk controls"[4]. A NeurIPS 2025 study of multi-agent systems documents failure rates between 41% and 87% across seven state-of-the-art frameworks, identifying 14 distinct failure modes with "no clear consensus on how to build robust systems"[5].

We need statistical confidence, not hope.

Declare what you want. Let evidence guide how it's built.

The first wave of AI applications was built by intuition: tweak a prompt, eyeball the output, ship it. This approach does not scale. It does not audit. It does not survive the handoff from researcher to production engineer. And it cannot keep pace with the combinatorial explosion that comes with multi-agent systems—where the configuration space spans models, tools, memory strategies, coordination protocols, and safety constraints that multiply faster than any team can reason about.

The second wave introduced automatic prompt optimization—pioneered by work like APE[6] and popularized by frameworks like DSPy[7] that search for better prompts given a metric and a dataset. This was progress: at least someone wrote down what "better" meant. But the search remained narrow (prompts only), the constraints implicit, and the promotion policy non-existent. You could find a better prompt, but you still shipped it on faith.

Recent research confirms what practitioners already feel: underspecification is endemic. A CMU study found that underspecified prompts are twice as likely to regress across model updates, sometimes with 20%+ accuracy drops[8]. An ICLR 2026 paper reports that 68% of "well-defined" SWE-Bench tasks were filtered out due to specification issues[9]. The problem is not careless prompt writing—it is the absence of a language for expressing what matters.

Today, we need the third wave: governed configuration for entire intelligent systems—not just prompts, but models, retrieval strategies, safety thresholds, and cost budgets, all optimized together under explicit constraints and statistical promotion gates.

[2] MIT Project NANDA, "The GenAI Divide: State of AI in Business 2025," 2025.
[3] S&P Global Market Intelligence, "Voice of the Enterprise: AI & Machine Learning," 2025.
[4] Gartner, Inc., Press Release, June 2025.
[5] Cemri et al., "Why Do Multi-Agent LLM Systems Fail?," NeurIPS 2025 Datasets and Benchmarks Track.
[6] Zhou et al., "Large Language Models Are Human-Level Prompt Engineers," ICLR 2023.
[7] Khattab et al., "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines," ICLR 2024.
[8] Liu et al., "Requirements-Aware Prompt Optimization," arXiv:2505.13360, 2025.
[9] "Interactive Agents to Overcome Underspecificity in Software Engineering," ICLR 2026.

TVL marks the transition from **agent building** to **agent engineering**—a shift as significant as the one that gave us software engineering itself. Building is craft; engineering is discipline. Engineering demands:

- **Specification** — a precise declaration of what the system should optimise and what it must never violate.
- **Verification** — automated checks that configurations satisfy constraints before they reach production.
- **Governed change** — promotion gates that require statistical evidence, not gut feeling, before a new configuration replaces the incumbent.

Separating What from How

Declarative programming taught us to separate *what* we want from *how* to compute it—SQL over loops, React over DOM manipulation. TVL extends this principle to the configuration of intelligent systems:

- **What**: "Maximise answer quality, minimise latency and cost, never exceed toxicity threshold 0.05."
- **How**: Let the optimizer explore the configuration space, respecting constraints, and propose candidates that dominate the current production baseline.

You declare the objectives and guardrails; the machine searches for the best configuration at this moment in time. When conditions change—new models, new prices, new policies—you do not rewrite code. You re-run the optimizer against the same specification and let evidence guide the next promotion. We stop tuning by hand; we run a governed loop.

Constraints are safety rails, not speed bumps: they accelerate progress by ruling out the impossible.

Tuned Variables: A New Primitive

Every paradigm shift in programming introduced a new primitive: procedures, objects, lambdas, promises. Each changed how we think about computation. The intelligent callable demands another: the *tuned variable*. You can only govern what you name.

A tuned variable is not a value you assign once. It is a *governed parameter* that is:

1. **Typed and bounded** — the spec declares its domain (enum, float range, integer set).
2. **Constrained** — structural rules prune infeasible combinations before any trial runs.
3. **Optimised** — a search algorithm explores the space, evaluating candidates against real workloads.
4. **Promoted** — only candidates that pass statistical gates replace the incumbent.

5. **Revisited** — when the environment drifts, the cycle repeats.

This lifecycle is what distinguishes a tuned variable from a configuration constant. Constants are set and forgotten; tuned variables are governed and evolved.

Why This Matters for Humanity

We stand at an inflection point. For the first time in history, we are building systems that *reason*—or at least approximate reasoning well enough to advise doctors, tutor students, write legal briefs, and negotiate contracts. These are not calculators. They are decision-making partners embedded in the critical infrastructure of society.

The stakes are too high for "it worked on my laptop." Too high for "the demo looked good." Too high for configuration by intuition and promotion by hope.

TVL is our answer: a specification language that makes AI configuration auditable, reproducible, and safe by design. It is how we ensure that the systems shaping human decisions are themselves governed by human values—expressed precisely, verified automatically, and evolved responsibly.

This is not just a technical contribution. It is a statement about how intelligent systems *should* be built: with the same rigour we demand of bridges, aircraft, and medical devices.

Welcome to the age of agent engineering.

— *Traigent Labs, 2025*

Contents

Preface **i**

1 Why TVL Exists **1**
1.1 LLM Pipelines Refuse to Sit Still . . . 1
1.2 Design Goals . . . 1
1.3 DRY Principle for Evaluation and Safety . . . 2
1.4 From Hard-Coding to Tuned Variables . . . 2
1.5 Running Example: Campus Orientation RAG . . . 3
1.5.1 Walking Through the Spec in Plain English . . . 5
1.6 How to Use This Book . . . 6
1.7 Field Notes Icons . . . 7

2 Getting Fluent in TVL **8**
2.1 First Contact . . . 8
2.2 A Minimal Working Spec . . . 8
2.3 Line-by-Line Walkthrough . . . 9
2.4 Typing Your Variables . . . 10
2.5 Validation Warm-Up . . . 10
2.6 Common Pitfalls . . . 11
2.7 Objectives and Budgets . . . 12
2.8 Promotion as a First-Class Citizen . . . 12
2.9 Interactive Lab: Orientation RAG Circuit . . . 12

3 Constraints, Units, and Safety Nets **13**
3.1 Why Units Matter . . . 13
3.2 Typed DNF Constraints . . . 13
3.3 Worked Example . . . 13
3.4 Constraint Playbook . . . 15
3.5 Interactive Drill: Latency Spike Preset . . . 15
3.6 Making Constraints Testable . . . 15
3.7 Debugging Failures Fast . . . 17

4 Patterns for Real Deployments **18**
4.1 Inheritance Without Surprises . . . 18
4.2 Staging vs. Production, Codified . . . 18
4.3 Constraint-Aware Overlays . . . 20
4.4 When to Extend vs. Duplicate . . . 20
4.5 Optimization Playbooks . . . 20
4.6 Hotfix Overlays . . . 21
4.7 Provenance and Promotion . . . 21

5 TVL Meets Triagent and DVL **22**
5.1 Unifying the Stack . . . 22
5.2 Deterministic Artifacts . . . 22
5.3 Handing Off to Triagent . . . 23
5.4 Lifecycle of a Promotion . . . 23
5.5 Interactive Scenarios . . . 23
5.6 DVL Keeps the Data Honest . . . 23
5.7 A Manifest That Glues It Together . . . 23
5.8 Automation Recipe . . . 24
5.9 TVL Tooling and Plug-ins . . . 25
5.10 Looking Ahead . . . 25

A Quick Reference and Resources **26**
A.1 Interactive Lab Access . . . 26
A.2 Compiling This Book . . . 26
A.3 Essential Commands . . . 26
A.4 Further Reading . . . 27
A.5 Exploring the Examples . . . 27
A.6 Toolchain Map . . . 27

Chapter 1

Why TVL Exists

1.1 LLM Pipelines Refuse to Sit Still

Large language model development today is notoriously under-specified. Prompts, retrieval depth, model choice, and safety filters often live in ad-hoc notebooks or scattered YAML fragments. When a researcher hands off a "working" configuration, the implicit assumptions rarely make it into production. The CAIN extended abstract reminds us that the world around our language model systems is volatile: model catalogs shift, latencies spike with traffic bursts, and price lists change without warning. Traigent's Tuned Variable Optimizer (TVO) frames this volatility as an engineering problem—the optimal configuration is not a single hard-coded value, but a moving target $v^*(E, I)$ conditioned on environment E and evaluation set I. Traditional configuration files cannot keep pace, so engineers either over-provision (wasting budget) or accept drift (sacrificing quality). TVL is born from that friction.

The Traigent Ecosystem

Three components work together:

- **TVL** (Tuned Variable Language) — the specification language you are learning in this book. It declares tuning variables, constraints, and objectives.
- **TVO** (Tuned Variable Optimizer) — the search engine that explores the configuration space defined by TVL, respecting constraints while optimizing objectives.
- **DVL** (Data Validation Language) — a companion language for defining evaluation datasets, quality metrics, and drift tests. DVL ensures that "answer quality" or "latency" means the same thing across projects and over time.

This book focuses on TVL, but you will see TVO and DVL referenced when we discuss optimization runs and evaluation pipelines.

1.2 Design Goals

TVL is a specification language that captures the semantics TVO needs to search safely:

- **Type-safe tuning variables** that distinguish categorical choices from continuous ranges.
- **Deterministic exports** so configuration can be generated, versioned, and reproducible.
- **Constraint grammar** (typed DNF) to encode business and safety rules alongside the numbers.
- **Governed evolution** with promotion gates, reviewers, and audit trails.
- **DRY evaluation and safety** through reusable constraint libraries and standardised metrics.

Instead of scattering this knowledge across spreadsheets, dashboards, and ad-hoc scripts, TVL centralises it so pipelines can adapt while staying well-behaved.

1.3 DRY Principle for Evaluation and Safety

Software engineers live by *Don't Repeat Yourself*. Yet LLM teams often copy-paste safety constraints across projects—toxicity thresholds, PII filters, latency caps—only to discover months later that one team silently relaxed a rule. TVL solves this with **import and composition**. A central `safety-baseline.tvl.yml` can define organisation-wide guardrails:

```
metrics.toxicity_score <= 0.05
metrics.pii_detected == false
```

Individual projects inherit these constraints, override only what they need, and get automatic updates when the baseline evolves. The same pattern applies to evaluation metrics: define a standard DVL suite once, import it everywhere, and guarantee that "answer quality" means the same thing across the campus chatbot and the HR assistant. This standardisation is not just tidy—it makes cross-project comparisons meaningful and audit trails tractable.

1.4 From Hard-Coding to Tuned Variables

Imagine maintaining an onboarding chatbot the night before the semester starts. A new model checkpoint drops, GPU prices wobble, and policy reviewers add a fresh refusal requirement. With hand-tuned prompt files you either ship blind or pull an all-nighter. TVL gives you a different move: express the feasible configuration space, tell TVO the goals, and let the optimizer search within the guardrails. When conditions shift again, you rerun the same playbook without rewriting application code. This reframing—LLM configuration as a governed, iterative artifact—is the core mindset change we want you to carry through the rest of the book.

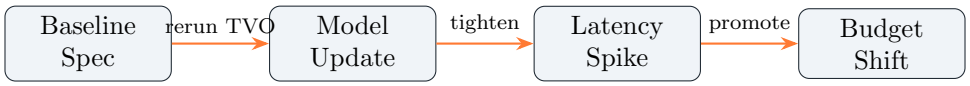

Figure 1.1: TVL encourages iteration: each environment change kicks off a repeatable optimization loop instead of a frantic patch.

1.5 Running Example: Campus Orientation RAG

Throughout the book we will revisit a retrieval-augmented generation (RAG) assistant that answers questions from incoming students. The evaluation set is quirky: it reflects traffic spikes during orientation week, budgets tighten after fiscal close, and latency promises must hold even when queries flood in at 8am. The TVL 0.9 spec below is our starting point.

Notice how the spec includes **safety constraints** (highlighted in red) alongside performance tuning. The `toxicity_score` and `pii_detected` guardrails ensure that no matter how aggressively TVO optimizes for quality or cost, the system never ships a configuration that could harm students. These safety rules are checked *before* any candidate reaches production—they are not afterthoughts bolted on during incident review.

Listing 1.1: TVL 0.9 spec for campus orientation RAG with safety constraints

```
# Campus Orientation RAG - TVL 0.9 Specification
# This example demonstrates type-safe tuning, safety constraints, and
    governed promotion.

tvl:
  module: campus.orientation.rag
tvl_version: "0.9"

# Environment snapshot anchors the spec to a point in time
environment:
  snapshot_id: "2025-01-15T00:00:00Z"
  components:
    model_catalog: us-east-1
    price_snapshot: Q1-2025

# Evaluation dataset for optimization trials
evaluation_set:
  dataset: s3://datasets/faq/onboarding-v2.jsonl
  seed: 2025

# Tuned Variables (TVARs) - the parameters we optimize
tvars:
  - name: model
    type: enum[str]
    domain: ["gpt-4o-mini", "gpt-4o", "claude-3-haiku"]

  - name: temperature
    type: float
    domain:
      range: [0.1, 0.8]
      resolution: 0.05

  - name: max_tokens
    type: int
    domain:
```

```
      set: [256, 512, 768]

  - name: retrieval_depth
    type: int
    domain:
      range: [2, 6]

# Constraints: structural (checked before trials) and derived (runtime)
constraints:
  structural:
    # Premium models need lower temperature for consistency
    - when: model = "gpt-4o"
      then: temperature <= 0.5
    # Deep retrieval requires more tokens
    - when: retrieval_depth >= 5
      then: max_tokens >= 512

  derived:
    # Runtime constraint on environment resources
    - require: daily_api_budget_remaining >= 10.0

  # SAFETY CONSTRAINTS - critical guardrails for student-facing use
  safety:
    - expr: toxicity_score <= 0.05
      error_message: "Responses must pass toxicity screening"
    - expr: pii_detected = false
      error_message: "No PII allowed in responses"

# Multi-objective optimization targets
objectives:
  - name: answer_quality
    direction: maximize
  - name: response_latency_p95_ms
    direction: minimize
  - name: token_cost_usd
    direction: minimize

# Promotion policy with statistical guarantees
promotion_policy:
  dominance: epsilon_pareto
  alpha: 0.05
  min_effect:
    answer_quality: 0.02
    response_latency_p95_ms: 50
    token_cost_usd: 0.005
  chance_constraints:
    - name: latency_slo
      threshold: 0.05
      confidence: 0.95
```

```yaml
# Exploration configuration
exploration:
  strategy:
    type: nsga2
  budgets:
    max_trials: 60
    max_wallclock_s: 2400
```

1.5.1 Walking Through the Spec in Plain English

Let's unpack each section so the terminology does not get in the way.

Header and Environment. The `tvl` block names the module; `tvl_version` pins us to the 0.9 spec. The `environment` section is a snapshot—it records *which* model catalog and pricing table were in effect when the spec was written. Think of it as a Git commit hash for external dependencies: if prices change tomorrow, you know which version your results came from.

Tuned Variables (tvars). These are the knobs TVO is allowed to turn. Each variable has a **type** and a **domain**:

- `model` is an *enum* (pick one from a list): `gpt-4o-mini`, `gpt-4o`, or `claude-3-haiku`.
- `temperature` is a *float* between 0.1 and 0.8, stepped by 0.05.
- `max_tokens` is an *int* from a fixed set: 256, 512, or 768.
- `retrieval_depth` is an *int* ranging from 2 to 6.

TVO will only ever propose combinations that fit these domains—no surprises like temperature = 5.0.

Constraints. Constraints are rules that *must* hold. TVL splits them into three flavours:

- **Structural** — checked before any trial runs. For example, "if you pick the expensive `gpt-4o` model, keep temperature at or below 0.5." These prune the search space so TVO never wastes time on forbidden combinations.
- **Derived** — checked at runtime against environment data. Here we require that `daily_api_budget_remaining >= 10.0`; if your budget is exhausted, the system blocks new trials instead of racking up charges.
- **Safety** — non-negotiable guardrails. Our example demands `toxicity_score <= 0.05` and `pii_detected = false`. Even if a configuration scores brilliantly on quality and cost, it is rejected if it fails safety.

Objectives. We want to *maximize* answer quality and *minimize* latency and cost. These three goals often conflict: a bigger model may boost quality but also increase cost. TVO does not collapse them into a single score; instead it searches for **Pareto-efficient** configurations—those where you cannot improve one objective without hurting another.

Plain-English aside: Imagine you are buying a laptop. You want long battery life, low weight, and a big screen. A Pareto-efficient laptop is one where you cannot get more battery without adding weight or shrinking the screen. TVO finds the set of such "best trade-off" configurations so you can pick the one that fits your priorities.

Promotion Policy. Before a new configuration replaces the current production one, it must pass the **promotion gate**. The policy here says:

- **epsilon_pareto** — the candidate must be measurably better, not just within noise. The `min_effect` block sets the bar: at least 0.02 improvement in quality, 50 ms faster, or $0.005 cheaper. Small random fluctuations do not count as wins.
- **alpha: 0.05** — statistical confidence level. We accept a 5% chance of being wrong when we declare the candidate better (the standard threshold in A/B testing).
- **chance_constraints** — even after passing the above, the candidate must satisfy probabilistic guarantees. Here, "latency_slo" says: with 95% confidence, no more than 5% of requests may breach the latency SLO. This protects against configurations that look good on average but have a long tail of slow responses.

Plain-English aside: Think of promotion policy as the rules for deploying a new microservice version. You would not ship code that passes unit tests but fails 10% of production traffic. The chance constraint is the performance equivalent: even if the median is great, the tail must behave.

Exploration. Finally, `exploration` tells TVO *how* to search:

- **strategy: nsga2** — a popular multi-objective genetic algorithm. It evolves a population of configurations, favouring those on the Pareto front.
- **budgets** — stop after 60 trials or 2400 seconds, whichever comes first. This caps compute cost and wall-clock time.

1.6 How to Use This Book

Each chapter mixes narrative, tables, and executable TVL listings stored under `tvl/tvl_book/examples`. Read them in order for a guided tour, or jump straight to the integration chapter if you already know the language and care about Triagent and DVL. Try editing the listings and running your own validation scripts—TVL is happiest when it powers real experiments.

1.7 Field Notes Icons

Keep an eye out for three recurring call-outs:

- **Hackathon Moves** highlight shortcuts for rapid prototyping.
- **Reliability Pitfalls** warn you when a choice carries hidden operational risk.
- **Integration Hints** show how TVL interacts with Triagent and DVL in practice.

They are sprinkled through later chapters to keep the pacing fun while grounding every concept in how we run real systems at Traigent Labs.

Chapter 2

Getting Fluent in TVL

2.1 First Contact

Pick up a TVL file and you should be able to answer three questions: what are we tuning, what does success look like, and who is accountable. Those answers live in different sections of the spec, and together they form a contract between engineers, operators, and the optimizer.

Section	Responsibilities
`spec`	Names the experiment and version so it can be tracked in git, DVC, and dashboards.
`metadata`	Declares owners, intent, and any review policy.
`configuration_space`	Declares the levers TVO can pull, along with their types and ranges.
`objectives`	Describes how we measure success and the direction we care about.
`optimization`	Configures the search strategy and budgets.
`constraints`	Encodes safety or business rules TVO must respect.
`evaluation`	Binds evaluation sets and metrics so automated tests remain grounded.
`promotion`	Governs the path from candidate to production.

Table 2.1: TVL sections capture different responsibilities in the tuning lifecycle.

2.2 A Minimal Working Spec

The quickstart listing below is intentionally small. Read it top to bottom: each section builds on the previous one, painted with YAML but backed by TVL's schema validations.

Listing 2.1: Minimal TVL specification from Chapter 1.1's simpler cousin.

```
spec:
  id: hello-tvl
  version: 0.0.1
metadata:
```

```
  owner: "students@traigent.ai"
  description: "Minimal TVL spec used in Chapter 2 to introduce the core
      sections."
  reviewers: ["mentor@traigent.ai"]
units:
  time: [milliseconds, seconds]
  ratio: [dimensionless]
configuration_space:
  model:
    type: categorical
    values: ["gpt-4o-mini", "claude-3-haiku"]
  temperature:
    type: continuous
    range: [0.0, 0.8]
    unit: ratio
  max_tokens:
    type: discrete
    values: [256, 384, 512]
    unit: tokens
objectives:
  - name: faithfulness
    direction: maximize
  - name: latency
    direction: minimize
optimization:
  strategy: pareto_optimal
  budget:
    max_trials:
      value: 30
      unit: count
    timeout_seconds: 900
constraints:
  - id: mild-temperature
    type: expression
    rule: 'params.temperature <= 0.8'
    error_message: "Stick to milder sampling in the quickstart."
evaluation:
  evaluation_sets:
    - name: tutorial
      dataset: dvl://demo/tutorial
promotion:
  gate: manual_review
  reviewers: ["mentor@traigent.ai"]
```

2.3 Line-by-Line Walkthrough

Start at the `spec` block and notice how the identifier, semantic version, and metadata form a paper trail. Down in `configuration_space`, we annotate the units so downstream services can display or

alert in the right scale. Each objective names the direction—`minimize` versus `maximize`—so you never wonder if lower latency is better (it usually is). Budgets give TVO the guardrails it needs to make smart choices without melting your GPU quota. Finally, promotion tells the human story: who reads the report before a change lands in production.

2.4 Typing Your Variables

TVL offers crisp typing for tuning variables:

- **Categorical** for discrete choices like model families.
- **Continuous** for floating-point ranges, optionally annotated with units.
- **Discrete** for enumerated integers that are meaningful as individual steps.
- **Integer** for ranged integers where the optimizer can choose any value between bounds.

This design keeps optimizers honest. For example, NSGA-II cannot accidentally propose half a retrieval depth, and gradient-based heuristics know which variables are safe to nudge smoothly.

2.5 Validation Warm-Up

Before we hand a spec to TVO we validate it locally. Chapter 2.1 ships with a tiny Python script that loads the YAML, checks for the fields we expect, and prints out the configuration space in a human-friendly table. The script lives beside the spec so you can extend it as you go.

Listing 2.2: Python helper that prints the configuration space and flags missing sections.

```python
"""Quick helper for Chapter 2 to inspect a TVL spec and print the knobs.
    """
from pathlib import Path
import textwrap

import yaml


def load_spec(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as handle:
        return yaml.safe_load(handle)


def ensure_sections(spec: dict, required: list[str]) -> list[str]:
    missing = [section for section in required if section not in spec]
    return missing


def print_configuration_space(space: dict) -> None:
    print("Configuration Space")
    print("-------------------")
```

```
    for name, definition in space.items():
        type_ = definition.get("type", "unknown")
        unit = definition.get("unit")
        details = []
        if "values" in definition:
            details.append(f"values={definition['values']}")
        if "range" in definition:
            details.append(f"range={definition['range']}")
        info = ", ".join(details)
        if unit:
            info = f"{info} ({unit})" if info else f"unit={unit}"
        print(f"- {name} [{type_}]: {info}")


def main() -> None:
    spec_path = Path(__file__).with_name("ch2_hello_tvl.tvl.yml")
    spec = load_spec(spec_path)

    required_sections = ["spec", "metadata", "configuration_space", "
        objectives", "optimization"]
    missing = ensure_sections(spec, required_sections)
    if missing:
        print("Missing sections:", ", ".join(missing))
        return

    print(textwrap.dedent(
        """\
        Spec loaded successfully.
        id       : {id}
        version  : {version}
        owner    : {owner}
        """
    ).format(
        id=spec["spec"].get("id", "<unknown>"),
        version=spec["spec"].get("version", "<unknown>"),
        owner=spec["metadata"].get("owner", "<unknown>"),
    ))

    print_configuration_space(spec["configuration_space"])


if __name__ == "__main__":
    main()
```

2.6 Common Pitfalls

- Forgetting units on continuous variables (a quick way to ship a 2-second timeout instead of 2 ms).

- Mixing integers and strings in a categorical list, which fails validation but wastes time.
- Leaving objectives unspecified, forcing TVO to guess what matters.
- Setting generous budgets with no promotion gate—the optimizer may explore forever without oversight.

Treat these as checklists before you commit a spec.

2.7 Objectives and Budgets

TVO defaults to multi-objective optimization, so list every metric you care about. If you only want a single target (say latency), keep the list to one entry. Budgets live alongside objectives to make the trade-off explicit: we spend computation in exchange for better configurations.

2.8 Promotion as a First-Class Citizen

Even a university hackathon deserves decent governance. A promotion gate such as `manual_review` or `strict_dominance` ensures there is a human or automated check before experiments impact users. Couple this with CI so merges fail if the gate denies the promotion. The TVL spec tells tools what to enforce.

2.9 Interactive Lab: Orientation RAG Circuit

The web companion to this book hosts a microsimulation that mirrors the quickstart spec. Visit the Orientation RAG lab (`../sims/orientation-rag-circuit/index.html`) to adjust the sliders and watch latency, cost, and quality gauges react in real time. Each slider writes back to the same fields taught in this chapter, while constraint badges map to the structural rules introduced in Chapter 3.1.

Offline readers. Scan the QR code in Appendix A.1 or open the bundled offline guide to follow along when you cannot load the web version.

Chapter 3

Constraints, Units, and Safety Nets

3.1 Why Units Matter

One of the sharpest critiques from the TVL unification review was that ambiguous fields invite human error. When a teammate reads `threshold: 2000` they should not have to wonder whether it is milliseconds or seconds. TVL therefore asks you to declare canonical unit domains and annotate variables accordingly. During export, tools like `params.yaml` append the unit to the key name (`cache_ttl_hours`) so downstream code is unambiguous.

3.2 Typed DNF Constraints

Constraints in TVL use a *typed Disjunctive Normal Form (DNF)* constraint language. It looks a little like Python expressions, but it is parsed and type-checked before each evaluation:

- **Expression constraints** evaluate to `true` or `false`.
- **Conditional constraints** evaluate a guard (`when`) and then enforce a rule.
- **Forbidden constraints** spell out anti-patterns you want to block outright.

The constraint language covers linear arithmetic, logic, membership tests, and interval bounds. Any constraint that returns something other than a boolean fails validation, making mistakes obvious.

3.3 Worked Example

The listing below demonstrates units, expressions, and conditional logic in one place. Open the file and try tweaking the numbers: what happens if you drop `cache_ttl_hours` below 1? How about pushing `rerank_weight` past 0.8? TVO will reject candidates that violate these rules before wasting a single trial.

Listing 3.1: Experiment exercising structural constraints and explicit units.

```
spec:
  id: constraints-lab
  version: 0.1.0
```

```yaml
metadata:
  owner: "qa@traigent.ai"
  description: "Explores CEL constraints and unit typing."
units:
  time: [milliseconds, seconds]
  currency: [USD]
  ratio: [dimensionless]
  percentage: [percent]
configuration_space:
  temperature:
    type: continuous
    range: [0.0, 1.0]
    unit: ratio
  max_tokens:
    type: integer
    range: [128, 1024]
    unit: tokens
  cache_ttl_hours:
    type: continuous
    range: [0.5, 12.0]
    unit: hours
  rerank_weight:
    type: continuous
    range: [0.0, 1.0]
    unit: ratio
objectives:
  - name: stability
    direction: maximize
  - name: cost
    direction: minimize
constraints:
  - id: cache-ttl-unit-check
    type: expression
    rule: 'params.cache_ttl_hours >= 1.0'
    error_message: "Cache needs at least 1 hour TTL."
  - id: token-temperature-budget
    type: expression
    rule: 'params.temperature * params.max_tokens <= 600'
    error_message: "Token-temperature product keeps hallucinations
       manageable."
  - id: rerank-mandate
    type: conditional
    when: 'params.max_tokens >= 512'
    then: 'params.rerank_weight >= 0.4'
    error_message: "Long answers must lean on reranking."
  - id: rerank-cap
    type: forbidden
    rule: 'params.rerank_weight > 0.8'
    error_message: "Rerank weight above 0.8 is unstable."
```

```yaml
evaluation:
  evaluation_sets:
    - name: stability_suite
      dataset: dvl://qa/stability-suite
  metrics:
    stability:
      aggregation: mean
    cost:
      aggregation: mean
```

3.4 Constraint Playbook

- Use **expression** constraints for universal truths (budget, latency, entropy caps).
- Reach for **conditional** constraints when a rule applies only to part of the space, such as premium models or weekend evaluation sets.
- Deploy **forbidden** constraints to codify "never again" lessons learned from incidents.

Compose them freely—the constraint language is designed to be readable on pager duty at 3am.

3.5 Interactive Drill: Latency Spike Preset

Load the Orientation RAG circuit lab's *Latency Spike* preset to watch constraint rules fire as you drag sliders. The latency meter mirrors the constraint

```
params.latency_ms <= 800
```

and the conditional rerank guard highlights the component whenever `retriever_top_k >= 40` while `rerank_weight < 0.3`. Use the exercise to build intuition before encoding the same constraints in YAML.

3.6 Making Constraints Testable

Because constraint rules are declarative, you can unit-test them just like code. A simple pattern is to load the spec in Python, feed sample parameter dictionaries, and assert that the constraint engine returns `true`. This keeps the rules alive as the configuration space evolves and guards against regressions when teammates add overrides.

Listing 3.2: Pytest-style checks that validate constraint expressions against sample assignments.

```python
"""Example pytest module that exercises the Chapter 3 CEL constraints."""
from __future__ import annotations

from pathlib import Path
from types import SimpleNamespace
from typing import Any, Dict
```

```python
import re

import yaml


SPEC_PATH = Path(__file__).with_name("ch3_constraints_units.tvl.yml")
SPEC = yaml.safe_load(SPEC_PATH.read_text(encoding="utf-8"))


def evaluate_constraint(expr: str, params: Dict[str, Any]) -> bool:
    """Tiny CEL-like evaluator for demonstration purposes."""
    python_expr = expr.replace("&&", " and ").replace("||", " or ")
    python_expr = re.sub(r"!(?!=)", " not ", python_expr)
    activation = {"params": SimpleNamespace(**params)}
    return bool(eval(python_expr, {}, activation))


def test_temperature_token_budget_passes() -> None:
    params = {
        "temperature": 0.5,
        "max_tokens": 512,
        "cache_ttl_hours": 2.0,
        "rerank_weight": 0.5,
    }
    expr = next(c["rule"] for c in SPEC["constraints"] if c["id"] == "
       token-temperature-budget")
    assert evaluate_constraint(expr, params)


def test_temperature_token_budget_fails() -> None:
    params = {
        "temperature": 0.95,
        "max_tokens": 768,
        "cache_ttl_hours": 3.0,
        "rerank_weight": 0.6,
    }
    expr = next(c["rule"] for c in SPEC["constraints"] if c["id"] == "
       token-temperature-budget")
    assert not evaluate_constraint(expr, params)


def test_rerank_requirement_triggers() -> None:
    constraint = next(c for c in SPEC["constraints"] if c["id"] == "rerank
       -mandate")
    params = {
        "temperature": 0.3,
        "max_tokens": 640,
        "cache_ttl_hours": 1.5,
        "rerank_weight": 0.2,
```

```
    }
    when_ok = evaluate_constraint(constraint["when"], params)
    then_ok = evaluate_constraint(constraint["then"], params)
    assert when_ok and not then_ok
```

3.7 Debugging Failures Fast

When TVO rejects a candidate it returns the failing constraint identifier. Pair that identifier with the `error_message` you wrote and log the parameter set. Most issues fall into one of three buckets:

- The override expanded a range without updating the constraint (narrow it or update the rule).
- A unit conversion went missing (milliseconds vs. seconds).
- The dataset changed shape and a derived metric (like `topic_overlap`) now has a different scale—time to revisit the DVL suite.

Capture the fix in Git history so future debuggers see the reasoning.

Chapter 4

Patterns for Real Deployments

4.1 Inheritance Without Surprises

TVL's `extends` and `overrides` mechanics turn sprawling configuration fleets into layered, auditable structures. The rules are simple but strict: you may narrow ranges, remove choices by taking subsets, and adjust thresholds, but you cannot change types or add new categorical values. This mirrors the validated design from the TVL unification document and prevents environment-specific overrides from drifting into a different schema altogether.

4.2 Staging vs. Production, Codified

The listing below captures a common dance: staging accepts wider experimentation, production tightens budgets and latency goals. Both inherit from the same base so operators can reason about differences line-by-line. Try diffing the two overlays to see how each environment tailors the knobs.

Listing 4.1: Environment overlays tighten ranges and thresholds without breaking type safety.

```
spec:
  id: overlays-fleet
  version: 0.2.0
metadata:
  owner: "platform@traigent.ai"
  description: "Shows how extends/overrides manage environments."
units:
  time: [milliseconds, seconds]
  ratio: [dimensionless]
  currency: [USD]
configuration_space:
  model:
    type: categorical
    values: ["gpt-4o-mini", "claude-3-sonnet", "claude-3-haiku"]
  temperature:
    type: continuous
    range: [0.2, 0.9]
    unit: ratio
```

```yaml
  max_tokens:
    type: integer
    range: [128, 1024]
    unit: tokens
  reranker:
    type: categorical
    values: ["e5-large", "voyage-2", "semantic-router"]
objectives:
  - name: helpfulness
    direction: maximize
  - name: latency
    direction: minimize
  - name: cost
    direction: minimize
optimization:
  strategy: pareto_optimal
  budget:
    max_trials:
      value: 80
      unit: count
    parallel_trials: 8
    timeout_seconds: 3600
environments:
  staging:
    overrides:
      configuration_space:
        model:
          values: ["gpt-4o-mini", "claude-3-haiku"]
        temperature:
          range: [0.2, 0.6]
      optimization:
        budget:
          max_trials:
            value: 30
            unit: count
  production:
    extends: staging
    overrides:
      configuration_space:
        max_tokens:
          range: [256, 768]
      objectives:
        - name: latency
          threshold:
            value: 750
            unit: milliseconds
      promotion:
        gate: strict_dominance
constraints:
```

```
  - id: premium-model-reranker
    type: conditional
    when: 'params.model == "claude-3-sonnet"'
    then: 'params.reranker in ["voyage-2", "semantic-router"]'
    error_message: "Premium models require smarter reranking."
triagent:
  pipeline: SupportChat
  deployment_targets: ["staging", "production"]
```

4.3 Constraint-Aware Overlays

Inside overrides you can encode stricter constraints for premium pathways or high-risk deployments. In the example, production insists on a 750 ms latency ceiling and restricts the long-form `max_tokens`. Because TVL validates the entire extended spec, these rules compose safely with base-level constraints.

4.4 When to Extend vs. Duplicate

If a new evaluation set shares 80% of its DNA with an existing spec, extend it. You inherit guardrails, unit domains, and metadata automatically. Reach for a fresh spec only when the configuration space is wildly different or the objectives no longer align. This discipline keeps your repository small and consistent, much like a well-factored codebase.

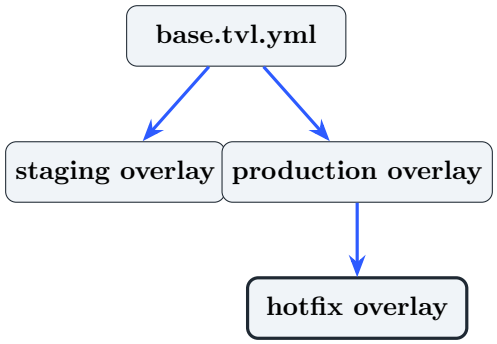

Figure 4.1: Layer overlays instead of copying specs. Hotfixes hang off production and are easy to remove later.

4.5 Optimization Playbooks

Multi-objective strategies like Pareto selection reward well-specified objectives. If you need a scalar score for ranking, consider the `display_weights` pattern from the integration design: keep weights only for presentation, not for the optimizer itself. This encourages healthy disagreement between objectives and surfaces trade-offs to reviewers instead of hiding them in a magic number.

4.6 Hotfix Overlays

Sometimes production needs a temporary constraint—a vendor outage, a compliance request, a surprise price hike. Instead of editing the base spec, create a short-lived overlay that narrows the affected ranges and annotate it with an expiration. Once the incident passes, delete the overlay and history will show exactly what changed.

Listing 4.2: Production hotfix overlay that temporarily caps premium model usage.

```
extends: overlays-fleet
metadata:
  owner: "sre@traigent.ai"
  description: "Temporary cap to avoid premium model saturation during
     vendor outage."
  expires_after: "2025-01-15"
overrides:
  configuration_space:
    model:
      values: ["gpt-4o-mini", "claude-3-haiku"]
    temperature:
      range: [0.2, 0.5]
  optimization:
    budget:
      max_trials:
        value: 25
        unit: count
  constraints:
    - id: temporary-cost-cap
      type: expression
      rule: 'params.cost_usd <= 0.025'
      error_message: "Hold cost under temporary ceiling during outage."
```

4.7 Provenance and Promotion

Real pipelines lean on deterministic exports (e.g. `params.yaml`, `tvl.lock`, `provenance.json`). Make sure your CI pipeline checks these files in alongside the TVL spec. When the promotion gate enforces strict dominance, TVO must prove that the new candidate beats the incumbent on every objective without bending constraints. The paperwork may feel formal, but it pays dividends when you audit a deployment months later.

Chapter 5

TVL Meets Triagent and DVL

5.1 Unifying the Stack

The refined integration plan positions TVL as the single source of truth, TVO (within Triagent) as the governed optimizer, and DVL as the data validation guardrail. The flow is deterministic: TVL defines the configuration space, TVO evaluates candidates and produces promotion proofs, and DVL verifies that the data feeding those experiments remains trustworthy.

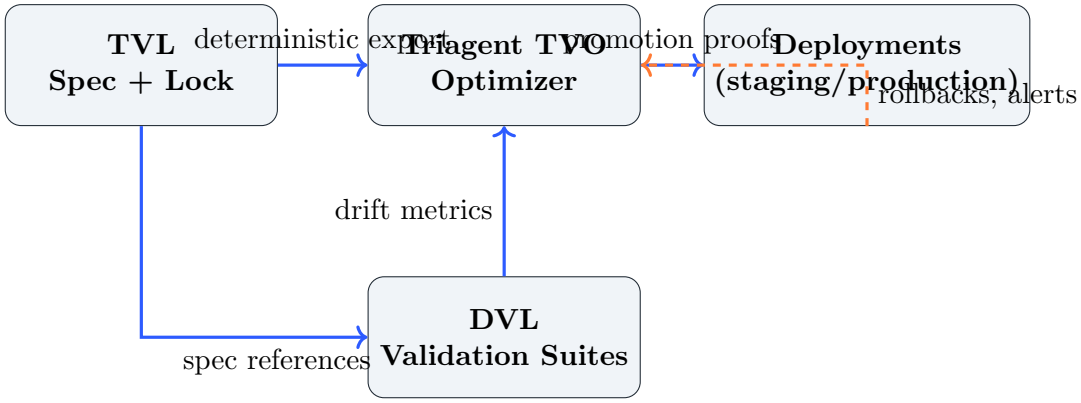

Figure 5.1: TVL anchors the configuration graph, Triagent/TVO executes governed optimization, and DVL feeds drift-aware validation signals.

5.2 Deterministic Artifacts

The integration spec insists that `params.yaml` remain deterministic, free of timestamps or noisy metadata. Instead, volatile fields move to `provenance.json`. This separation gives us reproducible runs, clean diffs, and clear audits. Always commit `tvl.lock`, `params.yaml`, and `provenance.json` together so reviewers see both the declarative intent and the resolved values.

5.3 Handing Off to Triagent

TVO reads the TVL spec directly, resolves overrides, and schedules evaluations. When a candidate dominates the incumbent on every objective (respecting constraints) the promotion gate flips. For staging environments, we often prefer manual review; production leans on `strict_dominance`. Triagent keeps the audit trail: who triggered the run, which seeds were used, and what rollback path is configured.

5.4 Lifecycle of a Promotion

1. Author updates the TVL spec and pushes it through code review.
2. CI calls `tvl validate` and `tvl export` to generate deterministic artifacts.
3. Triagent launches TVO with the resolved spec and recorded evaluation sets.
4. DVL validates datasets before and after the run, surfacing drift signals.
5. Promotion gate checks strict dominance and reviewer sign-off.
6. Deployment manifests (like Listing 5.1) capture the approved outputs.

Each step leaves a breadcrumb trail—logs, manifests, and audit records—so you can reconstruct decisions weeks later.

5.5 Interactive Scenarios

The Orientation RAG circuit lab ships with presets for the three incidents referenced here: baseline, latency spike, and budget relief. Replay them at `../sims/orientation-rag-circuit/index.html` to see how latency, cost, and quality trade-offs influence promotion decisions. The `Copy YAML` button mirrors the `params.yaml` export that feeds Triagent.

5.6 DVL Keeps the Data Honest

TVL captures the configuration; DVL certifies the datasets. Before promoting a configuration, Triagent invokes DVL suites to check for drift, PII leaks, or other regressions. Failed DVL checks block promotion even if the metrics look stellar. Treat DVL as equal peers in the pipeline: configuration without clean data is misleading, and clean data without tuned parameters under-delivers.

5.7 A Manifest That Glues It Together

Here is a manifest that a CI job could produce after a successful TVO run. It references the TVL spec, records the resolved SHA, and lists the validation jobs that must pass before deployment.

Listing 5.1: Deployment manifest binding TVL, Triagent, and DVL checks together.

```
# Generated deployment manifest linking TVL, Triagent (TVO), and DVL.
manifest:
  spec_path: tvl/spec/rag-campus-orientation.tvl.yml
  spec_sha: 7c2f31be
  tvl_lock: tvl.lock
  provenance: provenance.json
triagent:
  pipeline: RAGOrientation
  optimizer: TVO
  deployment_target: production
  evaluation_job: dvc repro deploy-rag-orientation
  promotion_gate: strict_dominance
  rollback_on:
    - latency_regression
    - constraint_violation
dvl:
  dataset_version: faq/v2024.10
  drift_tests:
    - faq-topic-shift
    - forbidden-pii
  validation_suite: dvl suites/faq-orientation.json
integration_checks:
  - tvl validate tvl/spec/rag-campus-orientation.tvl.yml
  - tvl export --lock tvl/spec/rag-campus-orientation.tvl.yml
  - dvl validate suites/faq-orientation.json
  - triagent deploy --dry-run RAGOrientation
```

5.8 Automation Recipe

The shell pipeline below mirrors what we run in CI: validate the spec, execute a dry run, run DVL suites, and collect the manifest. Use it as a template and extend it with notifications or rollback hooks.

Listing 5.2: Shell pipeline orchestrating TVL validation, Triagent dry run, and DVL checks.

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
SPEC="${ROOT_DIR}/ch1_motivation_experiment.tvl.yml"
MANIFEST="${ROOT_DIR}/ch5_integration_manifest.yaml"

echo "[validate] Checking spec..."
tvl validate "${SPEC}"

echo "[export] Producing deterministic artifacts..."
tvl export --lock "${SPEC}"
```

```
echo "[triagent] Dry-running pipeline..."
triagent deploy --dry-run RAGOrientation

echo "[dvl] Running validation suites..."
dvl validate suites/faq-orientation.json

echo "[manifest] Capturing manifest..."
cat "${MANIFEST}"
```

5.9 TVL Tooling and Plug-ins

Behind the `tvl` command is a constellation of plug-ins housed in `tvl_tools/`. Each module wraps a stage of structural or operational validation so you can mix them in CI or local workflows.

Module	Purpose
`tvl_parse`	Fast parser that ensures specs conform to the schema before any expensive checks.
`tvl_validate`	End-to-end validation including constraint evaluation, units, and override semantics.
`tvl_check_structural`	Lints inheritance graphs, duplicate IDs, and naming conventions.
`tvl_check_operational`	Evaluates runtime guardrails such as budget sizing and environment overlays.
`tvl_lint`	Style fixer for YAML layout, ordering, and comment hygiene.
`tvl_ci_gate`	Aggregates structural and operational checks for promotion pipelines.
`tvl_measure_validate`	Validates metric definitions and unit annotations referenced in specs.
`tvl_config_validate`	Validates CLI overrides and environment-specific diffs before deployment.

Table 5.1: Key plug-ins exposed through the `tvl` CLI. Pick and combine them in CI pipelines.

Most teams call these modules directly from Python or wrap them in `nox`/`tox` sessions. When Triagent assembles a promotion proof, it invokes `tvl_ci_gate`, collects reports, and attaches them to the manifest so reviewers see exactly which guardrails passed.

5.10 Looking Ahead

Future integrations will automate even more of this loop: Triagent can stream live metrics back into TVL, update `tvl.lock` with derived constraints, and share DVL drift annotations so engineers spot which evaluation sets deserve new objectives. Keep specs modular, keep units explicit, and let the tooling shoulder the boring bits so you can focus on designing better experiences.

Appendix A

Quick Reference and Resources

A.1 Interactive Lab Access

The TVL Intelligent Textbook ships with an Orientation RAG microsimulation that accompanies the walkthroughs in Chapters 2–5. When reading the web edition the lab is embedded inline; offline readers can launch it locally by opening `intelligent-textbooks/docs/sims/orientation-rag-circuit/index.` or by serving the site with `mkdocs serve` from `tvl/tvl_book/intelligent-textbooks`.

To regenerate scenario presets after editing a spec, run:

```
python -m tvl_tools.microsim_bridge path/to/spec.tvl.yml \
  --output tvl/tvl_book/intelligent-textbooks/docs/sims/orientation-rag-circuit/microsim_prese
```

This command snaps slider ranges to the configuration space, emits baseline plus incident presets, and records the spec identifier for provenance. Rebuild the MkDocs site or refresh the standalone HTML page to see the updates.

A.2 Compiling This Book

- Install a modern LaTeX distribution (TeX Live 2022+ recommended).
- From `tvl/tvl_book`, run `pdflatex main.tex` twice for stable references.
- Optional: add `biber` or `biblatex` if you extend the book with citations.

A.3 Essential Commands

- `tvl validate <spec>` — schema and constraint validation.
- `tvl export <spec> --lock` — produce deterministic `params.yaml` and `tvl.lock`.
- `triagent run <pipeline>` — execute TVO-driven optimization jobs.
- `dvl validate <suite>` — run dataset quality gates referenced in the spec.

A.4 Further Reading

- CAIN Extended Abstract: motivation for governed tuned variables.
- TVL Unification Design v2.0: architecture decisions and deterministic export rules.
- DVL Drift Playbook: recommended validation suites for common evaluation sets.

A.5 Exploring the Examples

- Chapter 1–2 specs live under `examples/ch1_*.tvl.yml` and `examples/ch2_*.tvl.yml`. Swap variable ranges and rerun the validation script to see error messages.
- Chapter 3 adds `ch3_constraint_tests.py`; run it with `pytest` to watch constraint rules fire.
- Chapter 4's overlays include `ch4_hotfix_overlay.tvl.yml`—practice removing it once a simulated incident ends.
- Chapter 5 bundles `ch5_integration_pipeline.sh`; adapt it to your CI runner.

A.6 Toolchain Map

The `tvl` CLI is modular. Each plug-in under `tvl_tools` can be invoked directly or composed in automation scripts. Use this cheat sheet when wiring them into CI.

Command	What it Checks
`python -m tvl_tools.tvl_parse <spec>`	Schema validation and fast parse reporting line-level issues.
`python -m tvl_tools.tvl_validate <spec>`	Full semantic validation, structural constraints, and units.
`python -m tvl_tools.tvl_check_structural <spec>`	Inheritance graph linting, duplicate detection, naming.
`python -m tvl_tools.tvl_check_operational <spec>`	Budget sanity, environment overlays, deployment readiness.
`python -m tvl_tools.tvl_lint <spec>`	Formatting, canonical key ordering, and comment cleanup.
`python -m tvl_tools.tvl_ci_gate <spec>`	Bundled gate used by Triagent promotion pipelines.
`python -m tvl_tools.tvl_measure_validate <metrics.yml>`	Ensures metric definitions and units match expectations.
`python -m tvl_tools.tvl_config_validate --override <file>`	Validates environment overrides or CLI patch files.
`python -m tvl_tools.microsim_bridge <spec>`	Generates Orientation RAG MicroSim presets for the interactive lab.

Table A.1: TVL plug-ins and the checks they perform. Combine them to match your deployment policies.

Acknowledgments

This book draws on the Traigent TVL specification, the CAIN extended abstract, and the TVL–TVO integration design. We thank the engineers, researchers, and reviewers who shaped the ecosystem—and who share our conviction that intelligent systems deserve the same rigour we demand of any critical infrastructure.